

1. FIRST PALINDROMIC STRING

```
Programiz C Online Compiler
main.c Run Output
#include <stdio.h>
#include <string.h>
int main() {
    int n = 5;

    char words[5][100] = {
        "abc",
        "car",
        "ada",
        "racecar",
        "cool"
    };

    for (int i = 0; i < n; i++) {
        int left = 0;
        int right = strlen(words[i]) - 1;
        int flag = 1;

        while (left < right) {
            if (words[i][left] != words[i][right]) {
                flag = 0;
                break;
            }
            left++;
            right--;
        }

        if (flag) {
            printf("%s\n", words[i]);
            return 0;
        }
    }

    printf("\n\n");
}
```

ada

=== Code Execution Successful ===

2. COUNT COMMON ELEMENTS

```
Programiz C Online Compiler
main.c Run Output
#include <stdio.h>
int main() {
    int n = 3, m = 2;

    int nums1[] = {2, 3, 2};
    int nums2[] = {1, 2};

    int answer1 = 0;
    int answer2 = 0;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (nums1[i] == nums2[j]) {
                answer1++;
                break;
            }
        }
    }

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            if (nums2[i] == nums1[j]) {
                answer2++;
                break;
            }
        }
    }

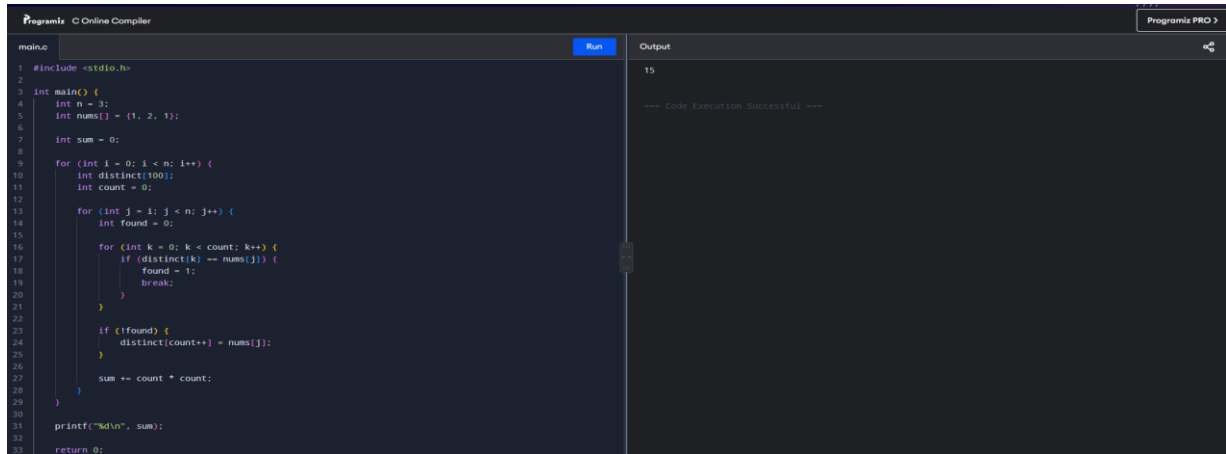
    printf("[%d,%d]\n", answer1, answer2);

    return 0;
}
```

[2,1]

=== Code Execution Successful ===

3. Sum of Squares of Distinct Counts

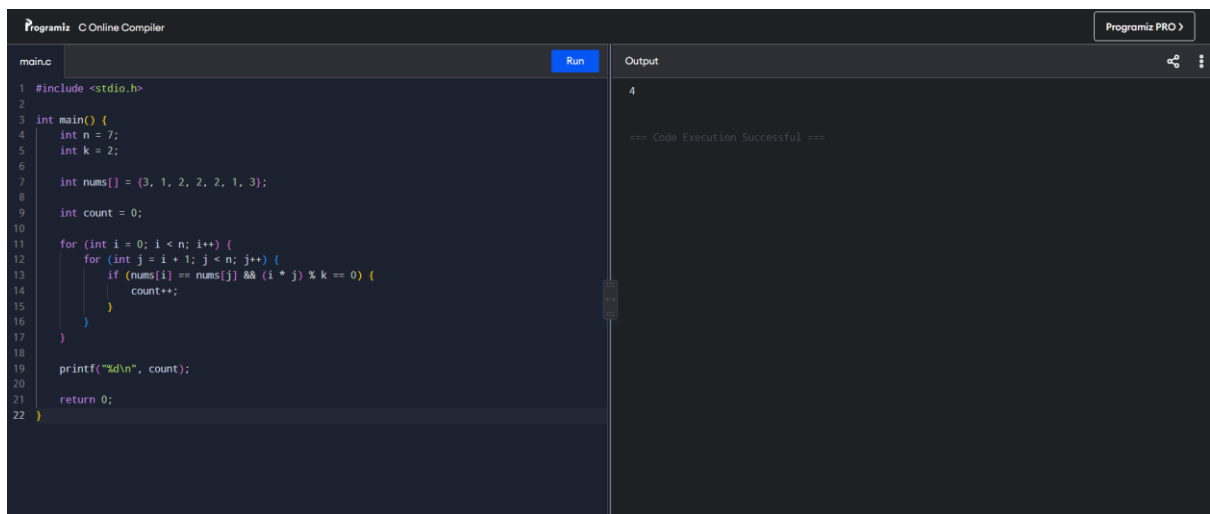


```
1 #include <stdio.h>
2
3 int main() {
4     int n = 3;
5     int nums[] = {1, 2, 1};
6
7     int sum = 0;
8
9     for (int i = 0; i < n; i++) {
10         int distinct[100];
11         int count = 0;
12
13         for (int j = i; j < n; j++) {
14             int found = 0;
15
16             for (int k = 0; k < count; k++) {
17                 if (distinct[k] == nums[j]) {
18                     found = 1;
19                     break;
20                 }
21             }
22
23             if (!found) {
24                 distinct[count++] = nums[j];
25             }
26
27             sum += count * count;
28         }
29     }
30
31     printf("%d\n", sum);
32
33     return 0;
34 }
```

Output: 15

--- Code Execution Successful ---

4. EQUAL ELEMENTS AND PRODUCT DIVISIBLE BY K

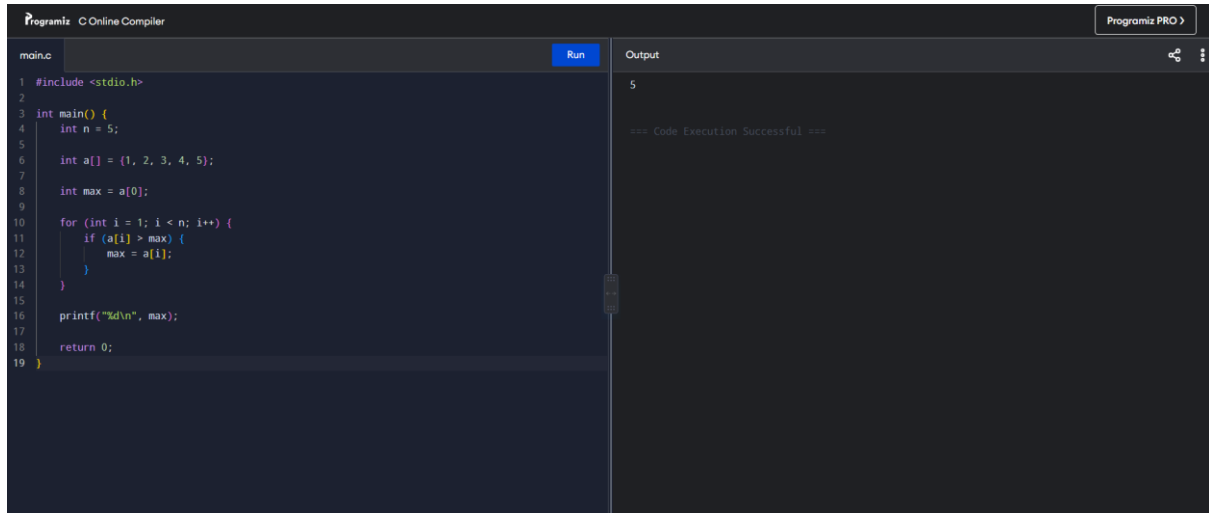


```
1 #include <stdio.h>
2
3 int main() {
4     int n = 7;
5     int k = 2;
6
7     int nums[] = {3, 1, 2, 2, 2, 1, 3};
8
9     int count = 0;
10
11     for (int i = 0; i < n; i++) {
12         for (int j = i + 1; j < n; j++) {
13             if (nums[i] == nums[j] && (i * j) % k == 0) {
14                 count++;
15             }
16         }
17     }
18
19     printf("%d\n", count);
20
21     return 0;
22 }
```

Output: 4

--- Code Execution Successful ---

5. FIND MAXIMUM ELEMENT



The screenshot shows a web-based C compiler interface. The left pane contains the source code for a program named 'main.c'. The code defines an array 'a' with values {1, 2, 3, 4, 5} and iterates through it to find the maximum value, which is stored in 'max'. The right pane shows the output of the program, which is the number '5'. Below the output, it says '=== Code Execution Successful ==='. The compiler's name 'Programiz C Online Compiler' is visible in the top left, and 'Programiz PRO' is in the top right.

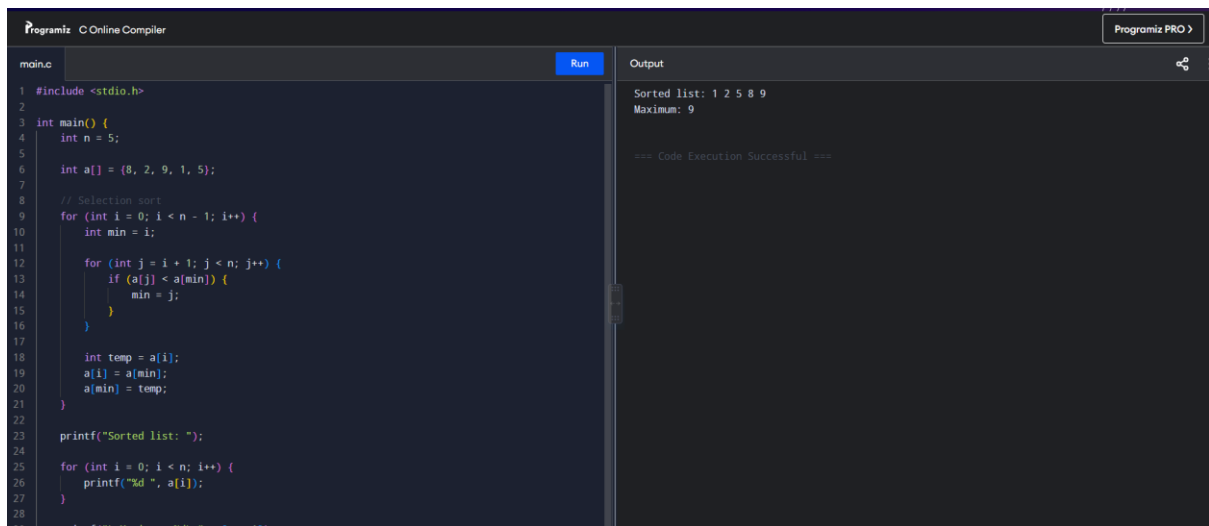
```
1 #include <stdio.h>
2
3 int main() {
4     int n = 5;
5
6     int a[] = {1, 2, 3, 4, 5};
7
8     int max = a[0];
9
10    for (int i = 1; i < n; i++) {
11        if (a[i] > max) {
12            max = a[i];
13        }
14    }
15
16    printf("%d\n", max);
17
18    return 0;
19 }
```

Output

5

=== Code Execution Successful ===

6. SORT LIST AND FIND MAXIMUM



The screenshot shows a web-based C compiler interface. The left pane contains the source code for a program named 'main.c'. The code defines an array 'a' with values {8, 2, 9, 1, 5} and implements a selection sort algorithm to sort the array. After sorting, it prints the sorted list and finds the maximum element, which is 9. The right pane shows the output: 'Sorted list: 1 2 5 8 9' and 'Maximum: 9'. Below the output, it says '=== Code Execution Successful ==='. The compiler's name 'Programiz C Online Compiler' is visible in the top left, and 'Programiz PRO' is in the top right.

```
1 #include <stdio.h>
2
3 int main() {
4     int n = 5;
5
6     int a[] = {8, 2, 9, 1, 5};
7
8     // Selection sort
9     for (int i = 0; i < n - 1; i++) {
10        int min = i;
11
12        for (int j = i + 1; j < n; j++) {
13            if (a[j] < a[min]) {
14                min = j;
15            }
16        }
17
18        int temp = a[i];
19        a[i] = a[min];
20        a[min] = temp;
21    }
22
23    printf("Sorted list: ");
24
25    for (int i = 0; i < n; i++) {
26        printf("%d ", a[i]);
27    }
28
29    printf("\nMaximum: %d\n", a[n - 1]);
30 }
```

Output

Sorted list: 1 2 5 8 9
Maximum: 9

=== Code Execution Successful ===

7. UNIQUE ELEMENTS

```
Programiz C Online Compiler
main.c Run
3 int main() {
4
5     int a[] = {3, 7, 3, 5, 2, 5, 9, 2};
6     int unique[8];
7
8     int count = 0;
9
10
11     for (int i = 0; i < n; i++) {
12         int found = 0;
13
14         for (int j = 0; j < count; j++) {
15             if (unique[j] == a[i]) {
16                 found = 1;
17                 break;
18             }
19         }
20
21         if (!found) {
22             unique[count] = a[i];
23             count++;
24         }
25     }
26
27     printf("Unique elements: ");
28
29     for (int i = 0; i < count; i++) {
30         printf("%d ", unique[i]);
31     }
32 }
```

Output

Unique elements: 3 7 5 2 9

== Code Execution Successful ==

8. BUBBLE SORT

```
Programiz C Online Compiler
main.c Run
1 #include <stdio.h>
2
3 int main() {
4     int n = 5;
5
6     int a[] = {5, 3, 4, 1, 2};
7
8     for (int i = 0; i < n - 1; i++) {
9         for (int j = 0; j < n - i - 1; j++) {
10
11             if (a[j] > a[j + 1]) {
12                 int temp = a[j];
13                 a[j] = a[j + 1];
14                 a[j + 1] = temp;
15             }
16         }
17     }
18
19     printf("Sorted array: ");
20
21     for (int i = 0; i < n; i++) {
22         printf("%d ", a[i]);
23     }
24
25     return 0;
26 }
```

Output

Sorted array: 1 2 3 4 5

== Code Execution Successful ==

9. BINARY SEARCH

```
Programiz C Online Compiler
main.c Run Output
2
3 int main() {
4     int n = 8;
5
6     int arr[] = {-9, 3, 4, 6, 8, 9, 10, 30};
7
8     int key = 10;
9
10    int low = 0;
11    int high = n - 1;
12
13    while (low <= high) {
14        int mid = (low + high) / 2;
15
16        if (arr[mid] == key) {
17            printf("Element %d is found at position %d\n",
18                key, mid);
19            return 0;
20        }
21        else if (arr[mid] < key) {
22            low = mid + 1;
23        }
24        else {
25            high = mid - 1;
26        }
27    }
28
29    printf("Element %d is not found\n", key);

```

Element 10 is found at position 6

=== Code Execution Successful ===

10. SORT ARRAY IN $O(N \log N)$ – MERGE SORT

```
Programiz C Online Compiler
main.c Run Output
1 #include <stdio.h>
2
3 void merge(int a[], int left, int mid, int right) {
4
5     int i = left;
6     int j = mid + 1;
7     int k = 0;
8
9     int temp[right - left + 1];
10
11    while (i <= mid && j <= right) {
12        if (a[i] < a[j])
13            temp[k++] = a[i++];
14        else
15            temp[k++] = a[j++];
16    }
17
18    while (i <= mid)
19        temp[k++] = a[i++];
20
21    while (j <= right)
22        temp[k++] = a[j++];
23
24    for (i = left, k = 0; i <= right; i++, k++)
25        a[i] = temp[k];
26 }
27
28 void mergeSort(int a[], int left, int right) {

```

Sorted array: 1 2 3 5 8

=== Code Execution Successful ===

11. OUT OF BOUNDARY PATHS

```
Programiz C Online Compiler
main.c Run Output
1 #include <stdio.h>
2
3 int dp[51][51][51];
4
5 int paths(int m, int n, int N, int r, int c) {
6
7     if (r < 0 || r >= m || c < 0 || c >= n)
8         return 1;
9
10    if (N == 0)
11        return 0;
12
13    if (dp[N][r][c] != -1)
14        return dp[N][r][c];
15
16    dp[N][r][c] =
17        paths(m, n, N - 1, r - 1, c) +
18        paths(m, n, N - 1, r + 1, c) +
19        paths(m, n, N - 1, r, c - 1) +
20        paths(m, n, N - 1, r, c + 1);
21
22    return dp[N][r][c];
23 }
24
25 int main() {
26
27     int m = 2;
28     int n = 2;
29     int N = 3;
30
31     printf("%d", paths(m, n, N, 1, 1));
32
33     return 0;
34 }
```

6

==> Code Execution Successful ==>

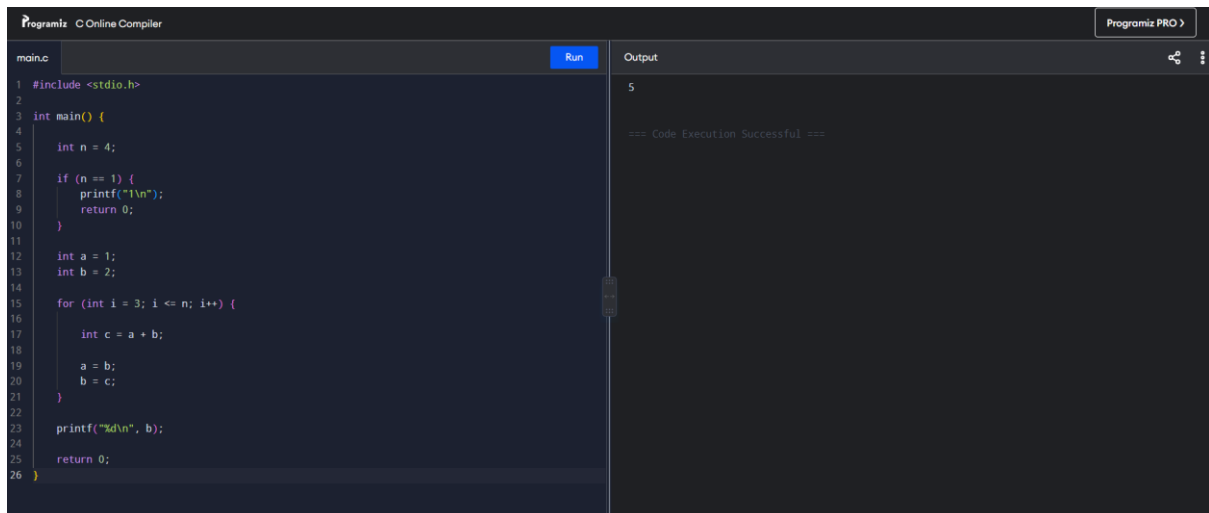
12. HOUSE ROBBER II

```
Programiz C Online Compiler
main.c Run Output
1 #include <stdio.h>
2
3 int maximum(int a, int b) {
4     return a > b ? a : b;
5 }
6
7 int rob(int nums[], int start, int end) {
8
9     int prev1 = 0;
10    int prev2 = 0;
11
12    for (int i = start; i <= end; i++) {
13
14        int current = maximum(prev1, prev2 + nums[i]);
15
16        prev2 = prev1;
17        prev1 = current;
18    }
19
20    return prev1;
21 }
22
23 int main() {
24
25     int n = 4;
26
27     int nums[] = {1, 2, 3, 1};
28
29     printf("Maximum money = %d", rob(nums, 0, n - 1));
30
31     return 0;
32 }
```

Maximum money = 4

==> Code Execution Successful ==>

13. CLIMBING STAIRS



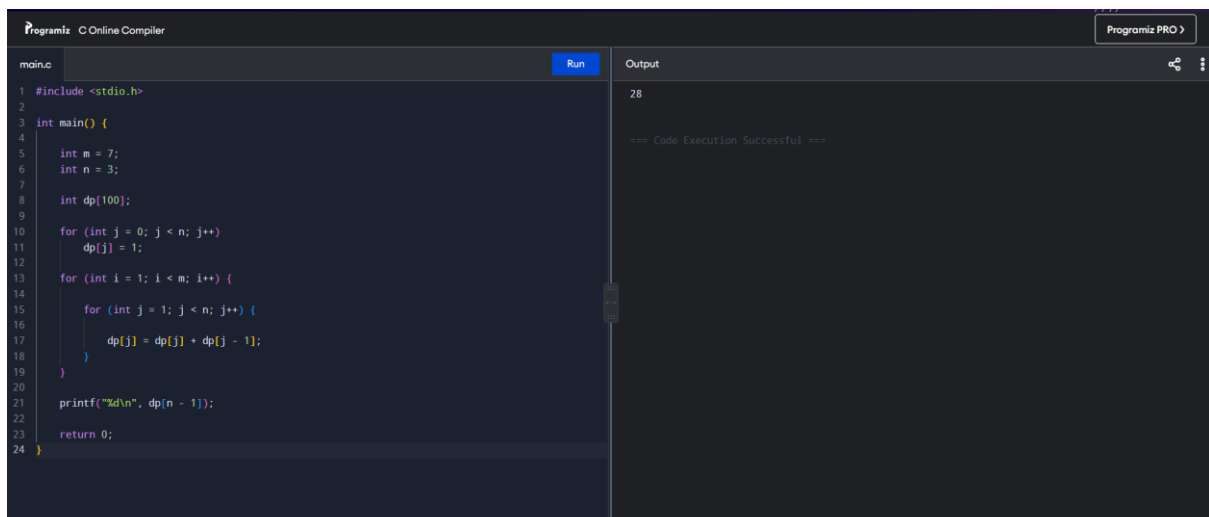
The screenshot shows the Programiz C Online Compiler interface. The code editor on the left contains a C program for calculating the number of ways to climb stairs. The output panel on the right shows the result of the program execution.

```
1 #include <stdio.h>
2
3 int main() {
4
5     int n = 4;
6
7     if (n == 1) {
8         printf("1\n");
9         return 0;
10    }
11
12    int a = 1;
13    int b = 2;
14
15    for (int i = 3; i <= n; i++) {
16
17        int c = a + b;
18
19        a = b;
20        b = c;
21    }
22
23    printf("%d\n", b);
24
25    return 0;
26 }
```

Output:

```
5
=== Code Execution Successful ===
```

14. UNIQUE PATHS



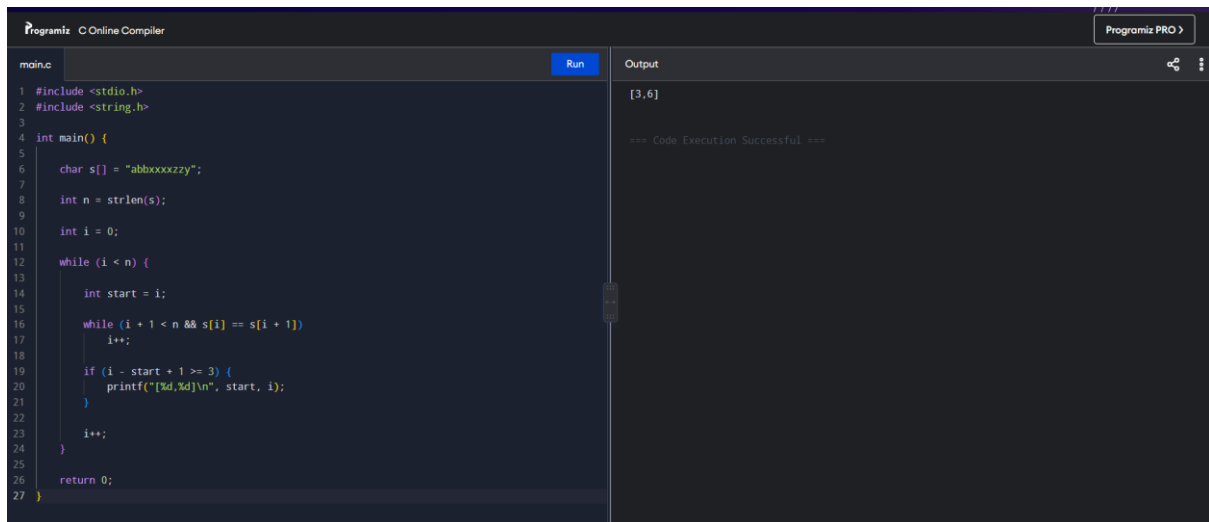
The screenshot shows the Programiz C Online Compiler interface. The code editor on the left contains a C program for calculating the number of unique paths in a grid. The output panel on the right shows the result of the program execution.

```
1 #include <stdio.h>
2
3 int main() {
4
5     int m = 7;
6     int n = 3;
7
8     int dp[100];
9
10    for (int j = 0; j < n; j++)
11        dp[j] = 1;
12
13    for (int i = 1; i < m; i++) {
14
15        for (int j = 1; j < n; j++) {
16
17            dp[j] = dp[j] + dp[j - 1];
18        }
19    }
20
21    printf("%d\n", dp[n - 1]);
22
23    return 0;
24 }
```

Output:

```
28
=== Code Execution Successful ===
```

15. LARGE GROUPS IN A STRING



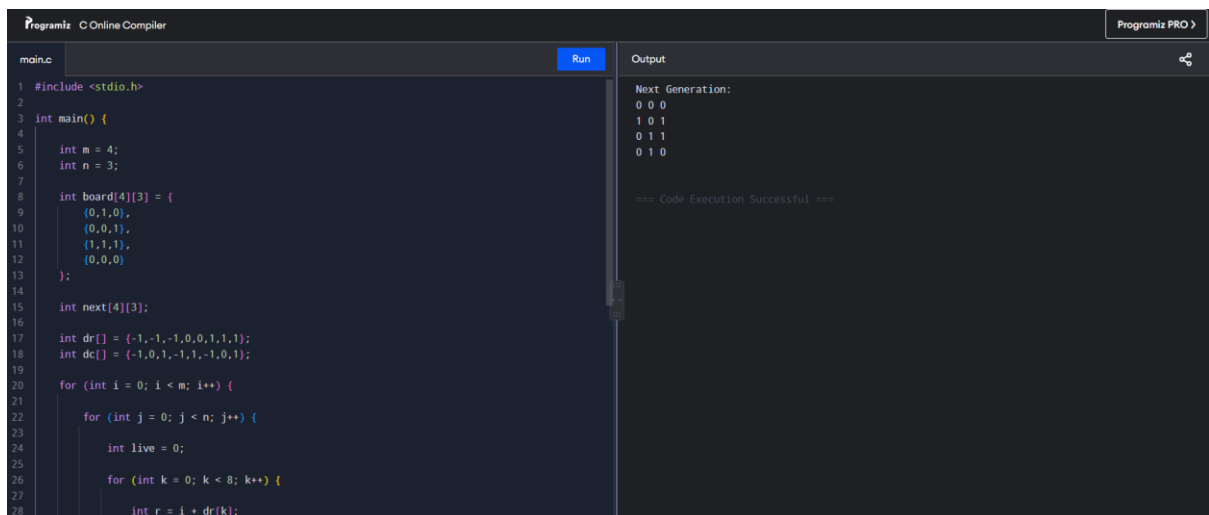
The screenshot shows a C program in an online compiler. The program defines a string `s` as "abbbxxxxzy". It iterates through the string, identifying groups of consecutive 'a's. The output shows the group [3,6], indicating a group of 3 'a's starting at index 6. The code execution is successful.

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5
6     char s[] = "abbbxxxxzy";
7
8     int n = strlen(s);
9
10    int i = 0;
11
12    while (i < n) {
13
14        int start = i;
15
16        while (i + 1 < n && s[i] == s[i + 1])
17            i++;
18
19        if (i - start + 1 >= 3) {
20            printf("[%d,%d]\n", start, i);
21        }
22
23        i++;
24    }
25
26    return 0;
27 }
```

Output: [3,6]

=== Code Execution Successful ===

16. GAME OF LIFE



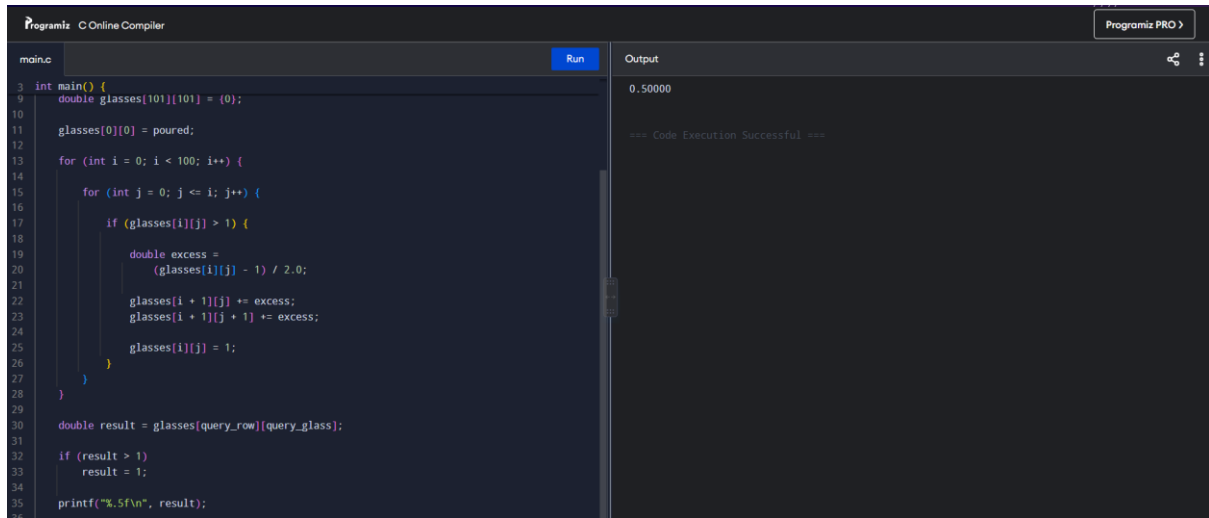
The screenshot shows a C program in an online compiler. The program initializes a 4x3 board with the following state: (0,1,0), (0,0,1), (1,1,1), (0,0,0). It calculates the next generation based on the rules of Conway's Game of Life. The output shows the next generation board: (0,0,0), (1,0,1), (0,1,1), (0,1,0). The code execution is successful.

```
1 #include <stdio.h>
2
3 int main() {
4
5     int m = 4;
6     int n = 3;
7
8     int board[4][3] = {
9         {0,1,0},
10        {0,0,1},
11        {1,1,1},
12        {0,0,0}
13    };
14
15    int next[4][3];
16
17    int dr[] = {-1,-1,-1,0,0,1,1,1};
18    int dc[] = {-1,0,1,-1,1,-1,0,1};
19
20    for (int i = 0; i < m; i++) {
21        for (int j = 0; j < n; j++) {
22            int live = 0;
23
24            for (int k = 0; k < 8; k++) {
25                int r = i + dr[k];
26                int c = j + dc[k];
27
28                if (r < 0 || r >= m || c < 0 || c >= n) continue;
29                live += board[r][c];
30            }
31
32            next[i][j] = (live == 3 || (live == 2 && board[i][j] == 1));
33        }
34    }
35
36    for (int i = 0; i < m; i++) {
37        for (int j = 0; j < n; j++) {
38            printf("%d ", next[i][j]);
39        }
40        printf("\n");
41    }
42
43    return 0;
44 }
```

Output: Next Generation:
0 0 0
1 0 1
0 1 1
0 1 0

=== Code Execution Successful ===

17. CHAMPAGNE TOWER



```
main.c Run Output
3 int main() {
9     double glasses[101][101] = {0};
10
11     glasses[0][0] = poured;
12
13     for (int i = 0; i < 100; i++) {
14         for (int j = 0; j <= i; j++) {
15             if (glasses[i][j] > 1) {
16                 double excess =
17                     (glasses[i][j] - 1) / 2.0;
18                 glasses[i + 1][j] += excess;
19                 glasses[i + 1][j + 1] += excess;
20                 glasses[i][j] = 1;
21             }
22         }
23     }
24
25     double result = glasses[query_row][query_glass];
26
27     if (result > 1)
28         result = 1;
29
30     printf("%.5f\n", result);
31 }
```

0.50000

== Code Execution Successful ==